

Mechduino manuál



Verze 1.1 – Duben 2025

Obsah

1	Úvod	3
2	Instalace	4
2.1	Instalace pomocí Matlab Apps	4
2.2	Instalační pomocí souboru	5
2.3	První spuštění	5
3	Mechduino	6
3.1	Připojení	6
3.2	pause	7
4	Arduino MEGA 2560	8
4.1	Nepájivé pole	8
5	Výstupní periferie	9
5.1	Logický výstup	10
5.1.1	LED	11
5.2	PWM výstup	13
5.3	Motory	15
5.3.1	DC motor	15
5.3.2	Krokový motor	20
5.3.3	Modelářské servo	22
6	Vstupní periferie	23
6.1	Logický vstup	24
6.1.1	Tlačítko	25
6.1.2	Enkodér	27
6.1.3	Sledování čáry	31
6.1.4	Indukční senzor	31
6.1.5	Senzor pohybu	32
6.2	Analogový vstup	33
6.2.1	Mikrofon	36
6.2.2	Senzor světla	36
6.3	Komunikační sběrnice	37
6.3.1	Akcelerometr + Gyroskop	37
6.3.2	Senzor hmotnosti	39
6.3.3	Magnetický enkodér	41
6.3.4	Senzor vzdálenosti - Lidar	42

6.3.5	Senzor vzdálenosti - Ultrazvuk	43
	Seznam obrázků	45
	Seznam zdrojů a použité literatury	113

1 Úvod

Mechduino - Výuková sada využívající programovací jazyk Matlab a používaná v předmětu *1PA - Úvod do programování a algoritmizace* pro otestování znalostí programování v prostředí Matlab na reálném HW v rámci závěrečného projektu předmětu.

Sada Mechduino je založena na vývojovém kitu Arduino Mega 2560, a několika vybraných čidlech a aktuátorů, jako jsou například DC a krokové motory, či senzory vzdálenosti a hmotnosti. Účelem sady je umožnění používání těchto prvků, bez hlubších znalostí jejich funkce a ovládání, pomocí jednoduchých příkazů z prostředí Matlab.

V tomto manuálu budou postupně představeny všechny prvky ze sady a popsány základy práce s nimi, jejich zapojení, ovládání a omezení.

Pro ovládání Arduina, a prvků připojených k němu prostřednictvím MATLABu, jsou vytvořeny třídy, jejichž instance představují konkrétní připojené prvky a voláním metod těchto instancí je možné tyto prvky ovládat. Všechny potřebné třídy budou v tomto manuálu představeny a v případě potřeby je pro tyto třídy v MATLABu vytvořena dokumentace. K dokumentaci pro konkrétní třídu se lze dostat prostřednictvím příkazu `help`, `doc` nebo stiskem tlačítka **F1**.

Všechny zde popsané třídy, kromě třídy **Mechduino**, jsou umístěny ve složce **controllers**. Pokud se budete chtít dostat k jejich dokumentaci prostřednictvím výše uvedených příkazů, musí být uveden pomocí tečkové notace název třídy spolu s jejím umístěním. Pokud se budete potřebovat dostat k dokumentaci např. třídy **Button** pomocí příkazu `doc`, budete muset do Command Window napsat následující příkaz:

```
doc controllers.Button
```

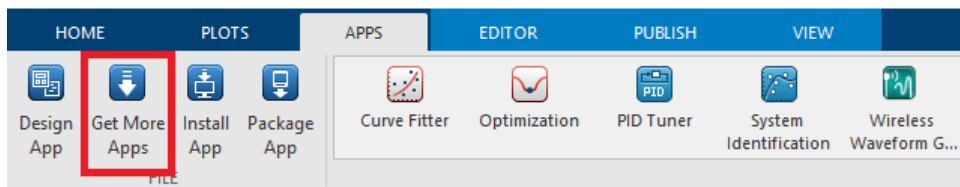

2 Instalace

Pro práci s toolboxem Mechduino je tento toolbox nutné nainstalovat do prostředí Matlab, to lze udělat dvěma způsoby. Prvním je instalace pomocí Matlab Apps, druhá je stáhnutím instalačního souboru.

2.1 Instalace pomocí Matlab Apps

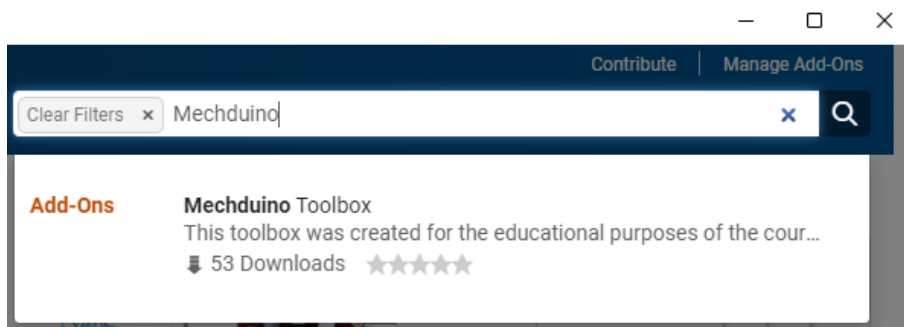
Návod vytvořen na základě Matlab 2024, pokud pro aktuální verzi nefunguje, dejte prosím vědět.

Pro instalaci doplňku přes Matlab je třeba spustit Add-ons explorer, ten lze nalézt v záložce APPS (obr. 2.1).

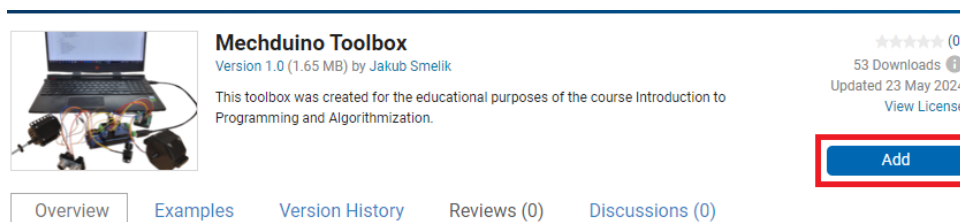


Obrázek 2.1: Menu aplikací v Matlab 2024a

V prostředí Add-ons explorer je pak nutné do vyhledávacího pole (vpravo nahoře, viz obr. 2.2) zadat "Mechduino", Následně dojde k přesměrování na stránku toolboxu a stačí jej přidat kliknutím na Add a odsouhlasením podmínek (obr. 2.3).



Obrázek 2.2: Vyhledání toolboxu Mechduino v Add-ons explorer



Obrázek 2.3: Instalace toolboxu Mechduino

2 INSTALACE

2.2 Instalační pomocí souboru

Instalační soubor Mechduina s názvem "Mechduino Toolbox.mltbx" lze stáhnout na Mechlabí dokuwiki, na stránkách kurzu 1PA:

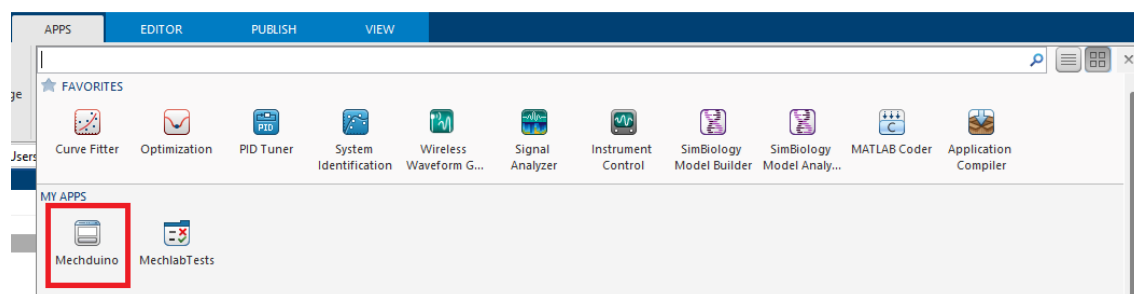
```
http://mechx.cz/dokuwiki/doku.php?id=courses:lpa:projekt
```

TODO: nahrát instalační soubor / přidat stránku na Dokuwiki

Stažený instalační soubor pak stačí jen spustit a automaticky dojde k nainstalování do Matlabu

2.3 První spuštění

Po instalaci toolboxu je třeba jednou spustit aplikaci Mechduino v panelu aplikací (obr. 2.4), čímž dojde k jejímu zavedení do systému.



Obrázek 2.4: Panel aplikací

Samotná práce s toolboxem je vysvětlena v následujících kapitolách.

3 Mechduino

Pro práci s jednotlivými periferiemi, které jsou popsány dále v manuálu, je třeba zavést instanci třídy Mechduino. Tato instance se zavádí pro každý připojený modul Arduina, tedy z jednoho počítače lze ovládat více zařízení najednou. Pro vytvoření instance třídy Mechduino je třeba znát název sériové komunikační linky, ke které je Arduino připojeno (obvykle značeno COMX, kde X je číslo daného připojení)

```
1      % Vytvoreni a ulozeni nove instance tridy Mechduino
2      Arduino_MEGA_mini = Mechduino('COM3');
```

3.1 Připojení

Pokud je na Arduinou nahrán ovládací program Mechduina, lze připojené zařízení detekovat pomocí následující funkce:

```
1      find_mechduino()
```

Výstup z funkce pak vypadá následně:

```
1      Zkousim port: COM1
2      Vystup z portu COM1: Zde nic
3      Zkousim port: COM3
4      Vystup z portu COM3: Mechduino tu je
5      Zkousim port: COM13
6      Vystup z portu COM13: Mechduino tu je
```

**Pozor, COM port je vázán na USB,
zapojením do jiného portu USB se změní číslo COM portu**

3 MECHDUINO

V případě, že na Arduinu není nainstalován program Mechduino (tedy zařízení se nezobrazí pomocí funkce "*find_mechduino()*"), je nutné port nalézt manuálně. Níže je popsán postup jak na to.

1. Připojíme Arduino do počítače a vypíšeme názvy dostupných COM portů, lze je zjistit pomocí následujícího příkazu:

```
1 serialportlist('available')
```

2. Pokud je dostupný pouze jeden COM port, jedná se o naše Arduino, pokud jich je dostupných více, název zařízení se nejjednodušeji zjistí následujícím postupem:

- odpojíme zařízení (Arduino)
- vypíšeme připojené zařízení (serialportlist)
- připojíme zařízení (Arduino)
- znovu vypíšeme připojené zařízení (serialportlist)
- nově připojené zařízení je naše Arduino

3. Vytvoříme novou instanci na nalezeném COM portu (vysvětleno na začátku kapitoly), program Mechduina se automaticky při prvním spuštění skriptu nahraje do zařízení.

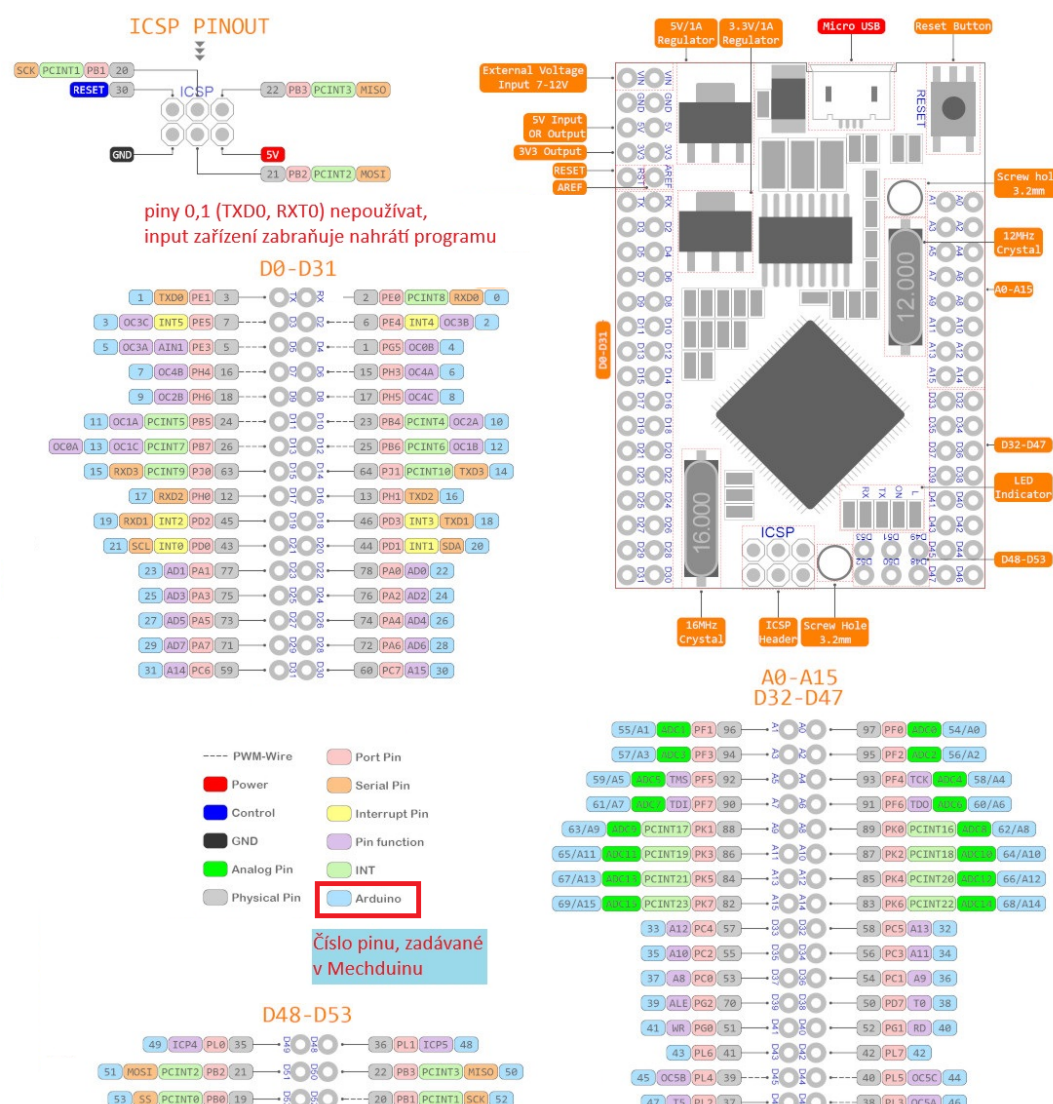
3.2 pause

Jelikož program ovládající periferie běží na Arduinu, ale řídicí program běží v prostředí Matlab na PC, základní metoda na pozastavení programu v prostředí Matlab "pause()" není dostatečná, neboť pozastaví pouze tu část programu, co běží na straně PC. Z toho důvodu je potřeba pro pozastavení celého programu použít funkci třídy Mechduino 'pause'. Doba pozastavení programu musí být zadána v milisekundách a je v rozsahu 0-65535.

```
1 % Pozastavení programu na 500 ms, tedy 0.5 sekundy
2 Arduino_MEGA_mini.pause(500);
```

4 Arduino MEGA 2560

Pro práci s třídou Mechduino je třeba Arduino vybavené procesorem 2560, to je v současné době Arduino MEGA a Arduino MEGA mini. Z důvodů větší kompaktnosti byla pro základní sadu stavebnice použita verze mini. Rozložení pinů je vidět na obrázku 4.1.



Obrázek 4.1: Popis Arduino MEGA mini 2560, upraveno z [1]

4.1 Nepájivé pole

má smysl popisovat? TBD

5 Výstupní periferie

Jako výstupní periferie lze souhrnně označit vše, co umožňuje zásah programu do reálného světa. Jako typické příklady výstupních periférií lze uvést kontrolku, která ukazuje vnitřní stav zařízení (nabíjení zařízení bez displeje), reproduktor poskytující akustickou zpětnou vazbu, nebo aktuátor, co něčím hýbe.

Pro dle způsobu zpracování byly výstupní periferie rozčleněny do tří kategorií:

1. Logický výstup

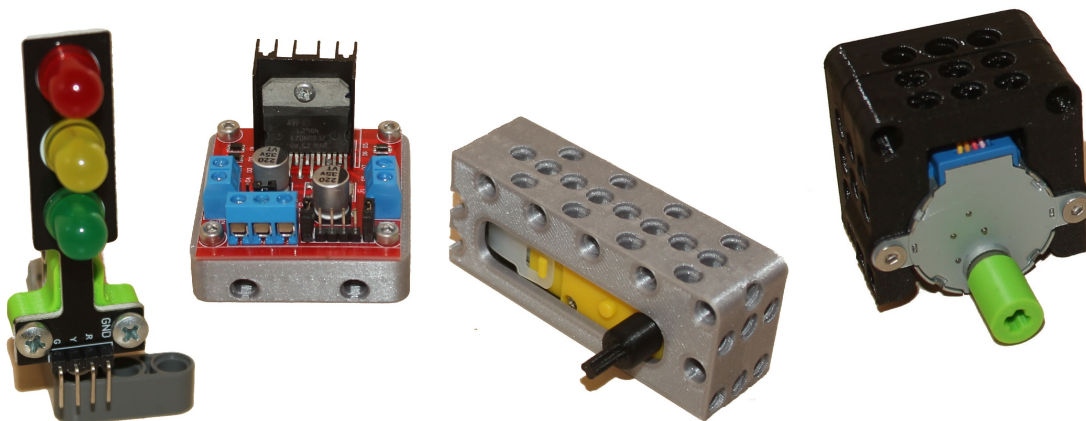
- výstupní hodnota 0/1, tedy boolean hodnota true/false
- typickým příkladem je LED dioda

2. PWM výstup

- někdy označován jako analogový výstup
- slouží k vytvoření průměrné hodnoty na výstupu v rozmezí 0 - U_{IN}
- v případě Arduina lze výstup nastavovat na 256 úrovní (0-255)

3. Motor

- periferie umožňující fyzickou interakci
- upraveno pro jednoduché ovládání
- Pro použití není vyžadována hlubší znalost funkce



Obrázek 5.1: Příklad výstupních periférií

5.1 Logický výstup

Logický výstup přivádí na vybraný výstupní pin logickou hodnotu false - 0 nebo true - 1. V praxi to znamená, že daný pin propojuje s GND pro logickou 0. Pro logickou 1 se na pin nastavuje hodnota napětí v rozsahu, se kterým dané zařízení pracuje. V případě Arduina se jedná o logiku TTL, kdy na pinu je napětí blízké vstupnímu, tedy 5 V.

Použití

Kdykoliv, když máme nějakou periférii, kterou ovládáme pomocí logických hodnot, příkladem můžou být:

- LED
- enable pin na periférii
- reset pin
- MOSFET tranzistor

Příklad

Pro demonstraci použijeme LED diodu, lze použít integrovanou LED diodu, která je připojena na pin 13.

```
1      % zdefinovani pinu, na kterem je LED pripojena
2      led_pin = 13;
3      % zdefinovani logickeho vystupu
4      led = Arduino_MEGA_mini.get_Logic_output(led_pin);
5      % nastaveni logicke 1 - true
6      led.set_hight();
7      % nastaveni logicke 0 - false
8      led.set_low();
```

Logický výstup má pouze tyto dvě funkce, nastavení na výstupu logické 0 a 1.

5 VÝSTUPNÍ PERIFERIE

5.1.1 LED

Pro přehlednější práci s LED diodami byla v mechduinu vytvořena vlastní třída.

```
1      % zdefinovani pinu, na kterem je LED pripojena
2      led_pin = 13;
3      % zdefinovani LED
4      led = Arduino_MEGA_mini.get_LED(led_pin);
5      % rozsviceni LED
6      led.light_up();
7      % zhasnuti LED
8      led.turn_off();
```

integrovaná led

Řídící deska Arduino Mega mini má na sobě integrovanou LED diodu připojenou kladným pólem na pinu 13. LED dioda je trvale připojena záporným pólem na GND. Přivedením logické 1 na pin 13 tedy dojde k jejímu rozsvícení a opačně přivedením logické 0 dojde ke zhasnutí. LED dioda se fyzicky nachází v rohu u pinů 41 a 43, na schématu (obr. 4.1) označena "L".

Modul LED semafor

Kromě integrované LED na desce je v sadě Mechduina modul LED semaforu (obr. 5.2), tento modul má 4 piny, a to společný pín "GND" a kladný pin pro každou LED, označené "R", "Y", "G". Modul lze připojit přímo do lišty na Mechduinu, jen je pak potřeba na pin "GND" přivést logickou 0.



Obrázek 5.2: modul LED semaforu

5 VÝSTUPNÍ PERIFERIE

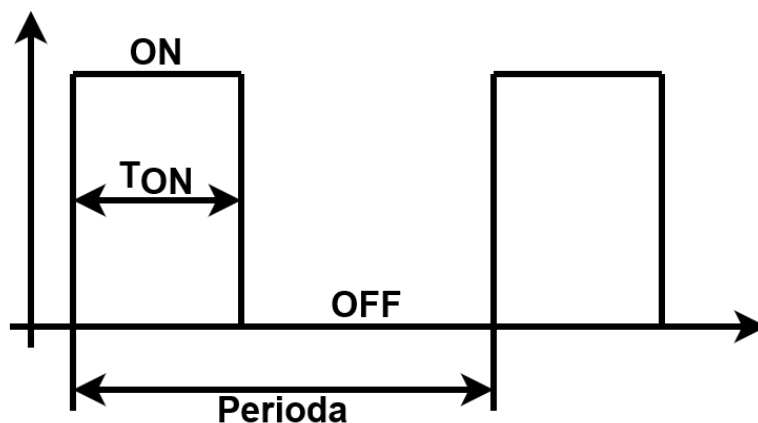
Při připojení přímo do modulu je třeba na pin "GND" přivést logickou nulu a pak jednotlivé LED rozsvěcet pomocí logické 1 na jejich pinech.

```
1      % LED semafor, pripojen na piny 2-8, kde GND je na
      pinu 2
2      ledGND = my_arduino.logic_output(2);
3      ledR = my_arduino.get_LED(4);
4      ledY = my_arduino.get_LED(6);
5      ledG = my_arduino.get_LED(8);
6
7      % je treba privest na GND logickou 0
8      ledGND.set_low();
9
10     % pak lze zapnout jednotlivé LED
11     ledR.light_up();
12     ledY.light_up();
13     ledG.light_up();
14
15     % TIP: vsechny LED lze najednou vypnout privedením
      logické 1 na GND
16     ledGND.set_high();
17
18     % po privedení logické 0 na GND se LED rozsvítí v
      konfiguraci, jak byly před vypnutím
19     ledGND.set_low();
```

5.2 PWM výstup

Pokud na výstupu nestačí pouhé přepínání mezi logickými hodnotami 1 a 0, je tato situace řešena některým ze způsobů převedení digitálního signálu na analogový. Nejjednoduším způsobem je PWM regulace, kdy na vysoké frekvenci (v řádu kHz) spínáme signál a modulací délky zapnutí (obr. 5.3) označované jako střída, zkráceně s (rovnice 5.1), nastavujeme na cílce plynule napětí v rozmezí $0 - U_{IN}$.

$$s = \frac{T_{ON}}{Perioda} \quad (5.1)$$



Obrázek 5.3: Popis PWM signálu

Počet napěťových hladin (tedy napětí, co můžeme na výstupu nastavit) je omezen velikostí PWM registru, který je v případě Arduina 8 bitový. Můžeme tedy nastavit 256 hodnot výstupního napětí (0-255), kde 0 je 0V a 255 je U_{IN} , výsledné napětí se pak počítá dle rovnice

$$U = s \cdot U_{IN} = \frac{[0; 255]}{255} \cdot 5 \text{ V} \quad (5.2)$$

Použití

PWM výstup je možné používat pouze na pinech 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 44, 45 a 46, Pokud není využívána některá jiná periferie, která by využití některých PWM pinů blokovala, třeba servo motor.

Kdykoliv, když máme nějakou periferii, kde chceme nastavit přesnou napěťovou hladinu, příkladem může být:

- LED
- Regulace motoru
 - pouze přes driver
 - malý proud na výstupu Arduina (100mA)
 - překročení proudu vede k poškození Arduina

5 VÝSTUPNÍ PERIFERIE

Příklad

Pro demonstraci použijeme LED diodu, kde budeme regulovat její jas, lze použít integrovanou LED diodu, která je připojena na pin 13.

```
1      % zdefinovani pinu, na kterem je LED pripojena
2      led_pin = 13;
3      % zdefinovani logickeho vystupu
4      led = Arduino_MEGA_mini.get_PWM_output(led_pin);
5      % pomale rozsveceni
6      for i = 0:255
7          led.PWM_set_s(i)
8          Arduino_MEGA_mini.pause(2)
9          % cas rozsveceni cca 0.5s (256x2 = 512 ms)
10     end
11     % pomale zhasnuti
12     for i = 255:-1:0
13         led.PWM_set_s(i)
14         Arduino_MEGA_mini.pause(2)
15     end
```

PWM output má pouze jednu funkci, kterou se nastavuje střída (0 - 255).

5.3 Motory

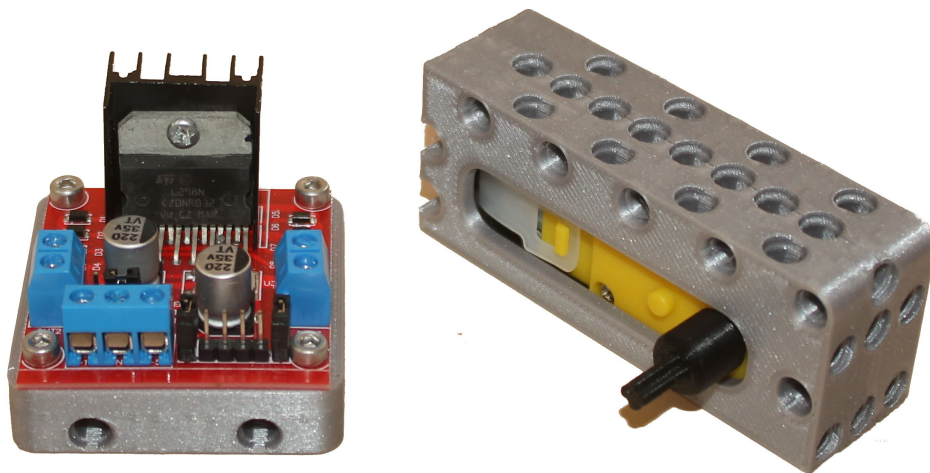
K fyzické manipulaci slouží motory, Knihovna Mechduino v současné podobě podporuje tři typy motorů:

- DC motor přes driver
- Krokový motor přes driver
- Modelářské servo

každý z těchto motorů je upraven pro snadné uživatelské ovládání, které je popsáno v následujících kapitolách.

5.3.1 DC motor

DC motor se používá do aplikací, kde je třeba regulovat rychlost, nebo po přidání dodatečných senzorů na regulaci polohy. V Mechduino sadě je v základu DC motor s převodovkou, který se připojuje přes driver L298N [2]. DC motor se připojuje pomocí dvou vodičů, polarita jejich zapojení udává směr otáčení a velikost napětí je úměrná rychlosti otáčení.

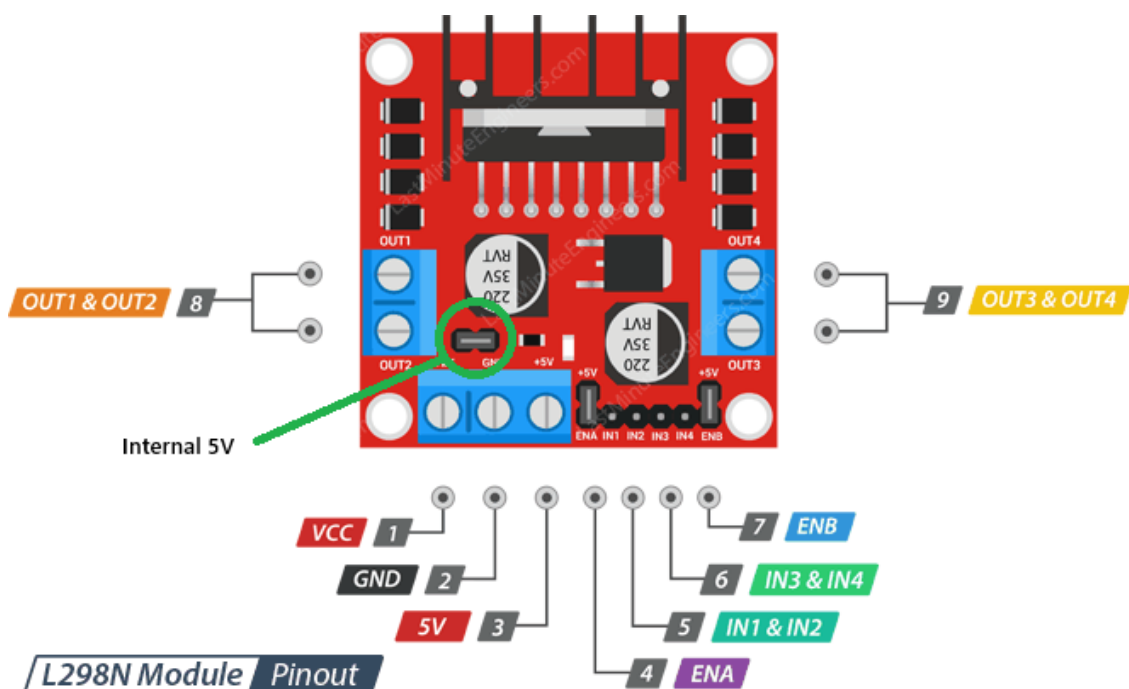


Obrázek 5.4: DC motor s driverem L298N

Driver L298N (obr. 5.5) má čtyři výstupy (OUT1 až OUT4), čtyři logické vstupy (IN1 až IN4) a enable piny (ENA a ENB) jenž zapínají jednotlivé porty (port A - IN1, IN2, OUT1, OUT2; port B - IN3, IN4, OUT3, OUT4), Piny ENA a ENB jsou v základu propojeny k logické 1 na desce pomocí zkratovacích propojek, takže jsou výstupy trvale zapnuté.

Motor je připojen jeho vývody na výstup OUT1 a OUT2 pro kanál A (OUT3 a OUT4 pro B). Propojení driveru s Mechduínem je možné třemi způsoby, pomocí jednoho, dvou nebo tří vodičů (obr. 5.6).

5 VÝSTUPNÍ PERIFERIE



Obrázek 5.5: Zapojení driveru L298N, převzato z [2]

• Jednovodičové připojení

Driver je připojen pomocí jednoho PWM pinu, je třeba dodržet, aby PWM pin byl na pinu umožňujícím generování PWM signálu (viz 5.2). Tyto piny se zapojují na IN1, IN2, IN3 a IN4. Motor je v tomto případě připojen ke driveru pouze jedním vodičem (pro IN1 na OUT1, ...) a druhý je připojen na GND.

– Výhody:

čtyři motory na driver

– Nevýhody:

motor lze pouze zabrzdit - v klidovém stavu klade aktivní odpor proti otáčení

nelze řídit směr otáčení, pouze rychlost

• Dvouvodičové připojení

Driver je připojen pomocí jednoho PWM pinu a jednoho logického pinu, je třeba dodržet, aby PWM pin byl na pinu umožňujícím generování PWM signálu (viz 5.2). Tyto piny se zapojují na IN1 a IN2 pro kanál A (IN3 a IN4 pro B).

– Výhody:

pouze dva vodiče

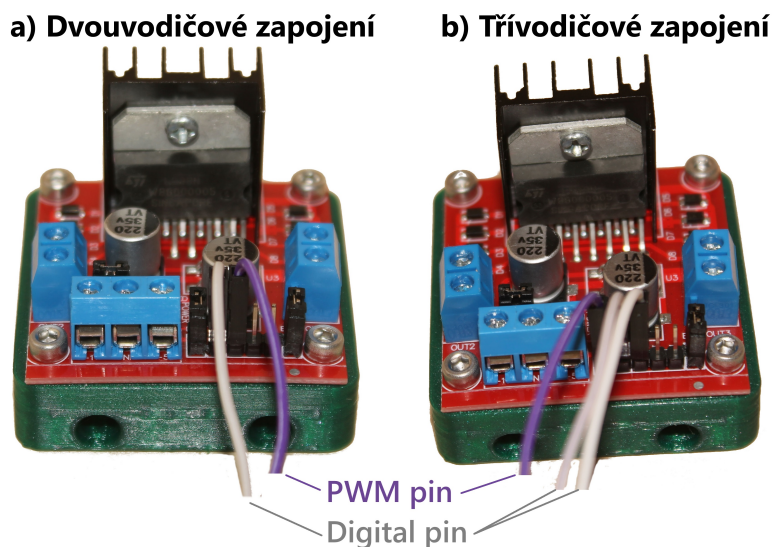
– Nevýhody:

motor lze pouze zabrzdit - v klidovém stavu klade aktivní odpor proti otáčení

• Třívodičové připojení

Driver je připojen pomocí jednoho PWM pinu a dvou logických pinů, je třeba

5 VÝSTUPNÍ PERIFERIE



Obrázek 5.6: Zapojení driveru L298N

dodržet, aby PWM pin byl na pinu umožňující generování PWM signálu (viz 5.2). Logické piny se zapojují na IN1 a IN2 pro kanál A (IN3 a IN4 pro B) a PWM pin se zapojuje na ENA (ENB).

- **Výhody:**
 - motor lze zastavit i zabrzdít
 - zastavení - otáčení brání pouze tření a svorné síly v převodu
 - zabrždění - v klidovém stavu klade aktivní odpor proti otáčení
- **Nevýhody:**
 - využívá tři vodiče

Použití

Limity použitého motoru a driveru:

- Maximální napětí na motoru = 9 V
- Maximální napětí na driveru = 35 V
- Maximální proud na driveru na kanál = 2 A
- Maximální výkon na driveru na kanál = 25 w
- Pro odběr více jak 0.5 A již nelze VCC připojit na 5 V Arduina

5 VÝSTUPNÍ PERIFERIE

Používané příkazy

Zavedení motoru pro jeden vodič:

```
1      % zadefinovani pinu, na kterem je motor pripojen
2      pinPWM = 4;
3      % PWM pins 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 44, 45, 46
4      motor = my_arduino.get_DC_motor(pinPWM);
```

Zavedení motoru pro dva vodiče:

```
1      % zadefinovani pinu, na kterem je motor pripojen
2      pinA = 28;
3      pinPWM = 4;
4      % PWM pins 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 44, 45, 46
5      motor = my_arduino.get_DC_motor(pinA, pinPWM);
```

Zavedení motoru pro tři vodiče:

```
1      % zadefinovani pinu, na kterem je motor pripojen
2      pinA = 28;
3      pinB = 30;
4      pinPWM = 4;
5      % PWM pins 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 44, 45, 46
6      motor = my_arduino.get_DC_motor(pinA, pinB, pinPWM);
```

Nastavení rychlosti:

pouze nastavuje rychlost otáčení, pokud je motor vypnutý neroztočí jej

```
1      speed = 138; % nastavena v tozmezi 0 - 255
2      motor.set_speed(speed);
```

Roztočení motoru:

pro nízké rychlosti se motor nemusí roztočit (nevyvine dostatečnou sílu k překonání suchého tření). Pro motor v sadě se tato hodnota pohybuje kolem 135.

```
1      % spusti motor s nastavenou rychlosti
2      motor.run();
```

5 VÝSTUPNÍ PERIFERIE

Zastavení motoru:

```
1      % zastavi a zabrzdí motor
2      motor.motor_brake();
3      % pouze zastavi motor
4      % funguje jen na 3 vodičové zapojení
5      motor.stop();
```

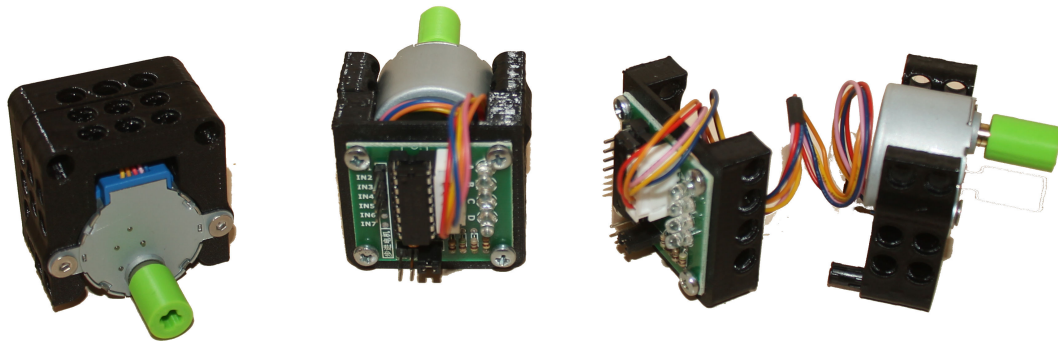
změna směru:

```
1      % obrátí směr otáčení motoru
2      % nefunguje pro 1 vodičové zapojení
3      motor.change_direction();
```


5 VÝSTUPNÍ PERIFERIE

5.3.2 Krokový motor

Krokový motor slouží k přesnému řízení polohy bez nutnosti dalších senzorů na snímání polohy (zpětné vazby). Krokový motor se otáčí o jednotlivé kroky, které jsou dány fyzickou konstrukcí motoru. V mechuino sadě je použit motor 28BYJ-48 [3] ovládaný pomocí driveru ULN2003 [4] (obr. 5.7). Driver se připojuje na čtyři digitální piny Arduina.



Obrázek 5.7: krokový motor s driverem

Používané příkazy

Zavedení motoru:

```
1      % zdefinovani pinu, na kterych je motor pripojen
2      p1 = 28;
3      p2 = 29;
4      p3 = 30;
5      p4 = 31;
6      stepper = my_arduino.get_Stepper(p1, p2, p3, p4);
```

Základním pohybem motoru je posun o jeden krok. Použité motory mají 2048 kroků na otáčku ($0,1755^\circ$, nebo $0^\circ 10' 33''$ na krok)

```
1      % jeden krůk v aktivním smeru
2      stepper.onestep();
3      % jeden krůk v základním smeru
4      stepper.onestep('Direction', 'Forward');
5      % jeden krůk v opačném smeru
6      stepper.onestep('Direction', 'Backward');
```

Pro řízení motoru je však řízení posunu po jednom kroku nevhodné a zdlouhavé,

5 VÝSTUPNÍ PERIFERIE

proto jsou implementovány dvě funkce, a to funkce na posun o X kroků a funkce na nastavení rychlosti posunu

```
1      % nastaveni rychlosti
2      stepper.set_speed(255); %0-255
3      % otacka v aktivnim smeru
4      stepper.step(2048);
5      % otacka v zakladnim smeru
6      stepper.step(2048, 'Direction', 'Forward');
7      % otacka v opacnem smeru
8      stepper.step(2048, 'Direction', 'Backward');
```

Aktivní směr lze otočit pomocí příkazu, který mění směr otáčení, nemá vliv na příkazy s parametrem 'Direction':

```
1      stepper.change_direction();
```

Pomocí následujících příkazů můžeme zjistit, zda se motor ještě točí, zastavit jej a znovu rozběhnout:

```
1      % zjisteni, zda se motor toci
2      stepper.is_moving()
3      % zastaveni motoru, pokud nema dokoncen soucasny
      pohyb
4      stepper.stop
5      % dokonceni pohybu po prikazu .stop
6      stepper.m_continue
```

5 VÝSTUPNÍ PERIFERIE

5.3.3 Modelářské servo

Modelářské servo je poslední integrovanou možností pohybu v knihovně Mechduino. v knihovně je integrovaná podpora pro obvyklá modelářská serva (obr. 5.8 řízené šířkou řídicího pulsu v rozsahu 0.5 až 2.5 ms (obvykle 0 – 180 °).



Obrázek 5.8: modelářský servo motor, převzato z [5]

Modelářské servo je vhodné použít v případě, kdy je třeba v malém rozsahu něco polohově řídit (poloha klapky, otáčení předmětu o malý úhel, přepínání cest při třídění něčeho, mačkání fyzického tlačítka, ...).

**Napájet z Arduina lze pouze jedno servo,
pro připojení více serv je nutné využít extení napájení**

Používané příkazy

Zavedení serva:

```
1      % zdefinovani pinu, na kterych je servo pripojeno
2      pin = 16;
3      stepper = my_arduino.get_Servo(pin);
```

Nastavení úhlu (v rozmezí 0 – 180°, *nebo* 0 – π rad):

```
1      % uhel je mozno nastavit ve stupnich
2      servo.set_angle_deg(135);
3      % nebo v radianech $
4      servo.set_angle_rad(0.25);
```


6.1 Logický vstup

Logický vstup je nejjednodušší vstupní periferie, která rozlišuje v základu pouze dva stavy, a to logickou 1 (zapnuto, true) a logickou 0 (vypnuto, false). Používá se kdekoliv, kde je možné vstup rozdělit do dvou stavů (zapnuto/vypnuto, svítí/nesvítí, pohyb/bez pohybu), nebo máme dopředu stanovenou pevnou hodnotu nad kterou chceme na data reagovat (napětí je větší než X V, intenzita hluku je nad XX dB, vzdálenost je menší než X cm, ...).

Použití

Kdykoliv, kde je možné vstup rozdělit do dvou stavů, nebo máme dopředu stanovenou pevnou hodnotu při které chceme na data reagovat, příkladem můžou být:

- přepínač (zapnuto / vypnuto)
- senzor pohybu (PIR, doppleruv senzor)
- indukční senzor (koncový spínač)
- sledování čáry (senzor odrazivosti)
- sledovat vybití / nabití baterie (napětí je $</>$ než V_{MIN}/V_{MAX})
- kontrolovat maximální intenzitu hluku (hluk nepřekročí X dB)
- soumračný senzor (intenzita světla je menší než X lux)

Používané příkazy

Zadefinování logického vstupu:

```

1      % zadefinovani pinu
2      sensor_pin  = 20;
3      % zadefinovani logickeho vstupu
4      sensor = my_arduino.get_Logic_input(sensor_pin);

```

Zjištění vstupní hodnoty:

```

1      % vraci true/false dle hodnoty na vstupu
2      sensor.read()

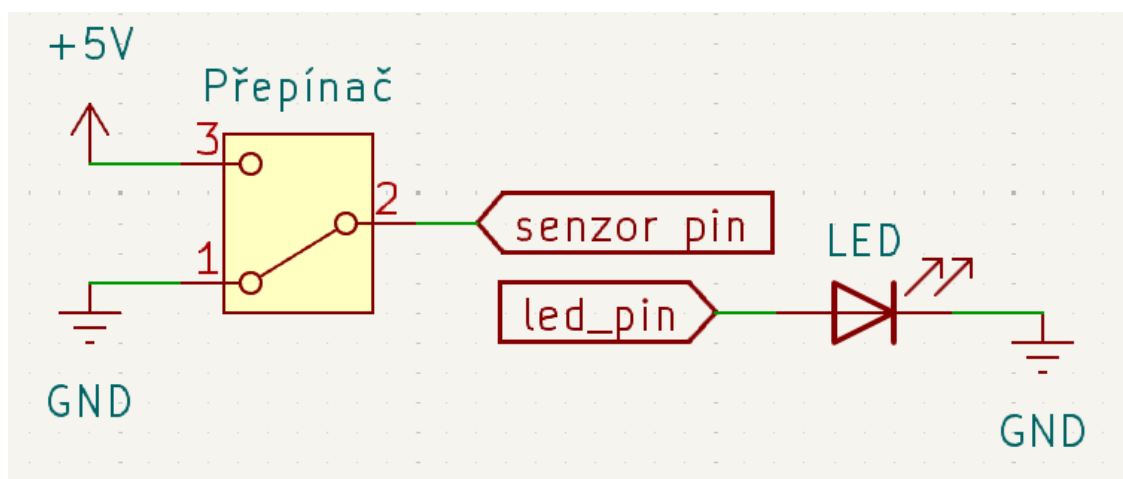
```

Příklad

Pro demonstraci použijeme přepínač, jako logický vstup a LED diodu pro vizuální kontrolu, lze použít integrovanou LED diodu, která je připojena na pin 13. V příkladu LED dioda zobrazuje hodnotu na logickém vstupu po dobu 30 s. (schéma zapojení na obr. 6.2)

6 VSTUPNÍ PERIFERIE

```
1      % zdefinovani pinu senzoru a LED
2      sensor_pin = 20;
3      led_pin = 13;
4      % zdefinovani LED a logickeho vstupu
5      sensor = my_arduino.get_Logic_input(sensor_pin);
6      led = Arduino_MEGA_mini.get_LED(led_pin);
7
8      t = tic;
9      while(toc(t) < 30)
10         if(sensor.read())
11             led.light_up();
12         else
13             led.turn_off();
14
15         end
16     end
```



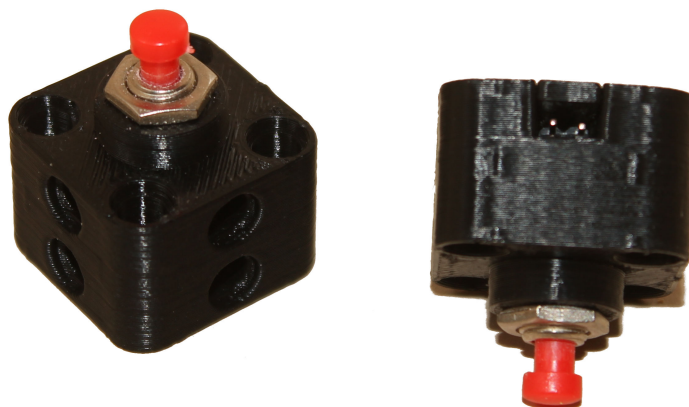
Obrázek 6.2: Schéma zapojení logického vstupu

6.1.1 Tlačítko

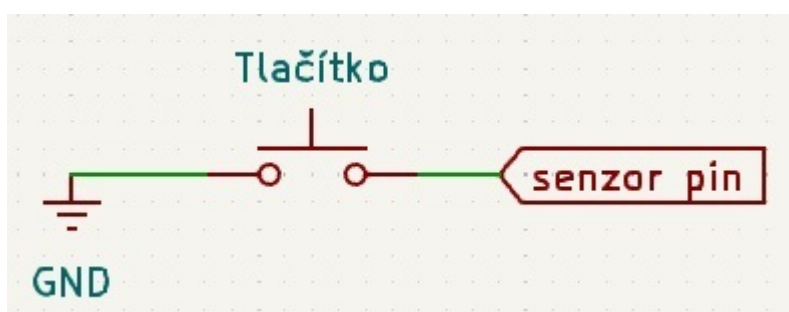
V případě použití tlačítka (obr. 6.3) by v logickém vstupu nastal problém při jeho rozpojeném stavu, pin by byl tou dobou v nezadefinovaném stavu a mohlo by se

6 VSTUPNÍ PERIFERIE

na něm v tu chvíli vyskytovat logická 1, 0 nebo docházet k oscilaci mezi těmito hodnotami. Aby k této situaci nedocházelo, je nutné tlačítko vybavit pull-up, nebo pull-down rezistorem, které se starají o nastavení předdefinované hodnoty pro rozpojený stav. Arduino má vnitřní pull-up rezistor, tlačítko se tedy zapojuje mezi něj a GND (schéma zapojení na obr. 6.4).



Obrázek 6.3: Tlačítko v sade Mechduino



Obrázek 6.4: Schéma zapojení tlačítka

Zadefinování logického vstupu:

```
1  % zdefinovani pinu
2  sensor_pin = 20;
3  % zdefinovani logickeho vstupu
4  button = my_arduino.get_Button(sensor_pin);
```

Zda je tlačítko zmáčkuté se zjišťuje pomocí následujícího příkazu:

```
1  % vraci true/false dle hodnoty na vstupu
2  button.is_pushed()
```

6 VSTUPNÍ PERIFERIE

Pro případy, kdy chceme zjistit, zda tlačítko bylo zmáčknuté v době, kdy se nekontroloval stav, existují dva příkazy, které kontrolují zda došlo ke změně stavu. Kontroluje se zda se v signálu objevila náběžná hrana (zmáčknuté tlačítko bylo uvolněno), nebo sestupná hrana (zmáčknutí tlačítka) (náčrtek na obr. 6.5).

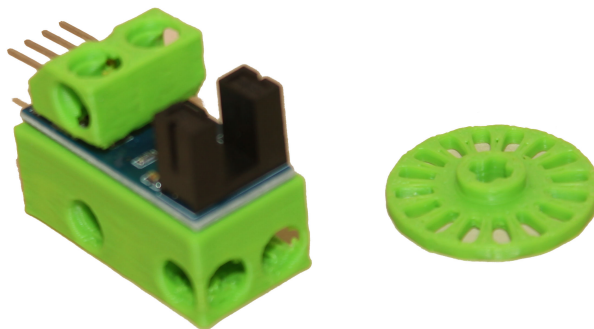
```
1 % vraci true/false dle hodnoty na vstupu
2 button.get_rising_edge() % kontrola nabezne hrany
3 button.get_falling_edge() % kontrola sestupne hrany
```



Obrázek 6.5: Náběžná a sestupná hrana signálu

6.1.2 Enkodér

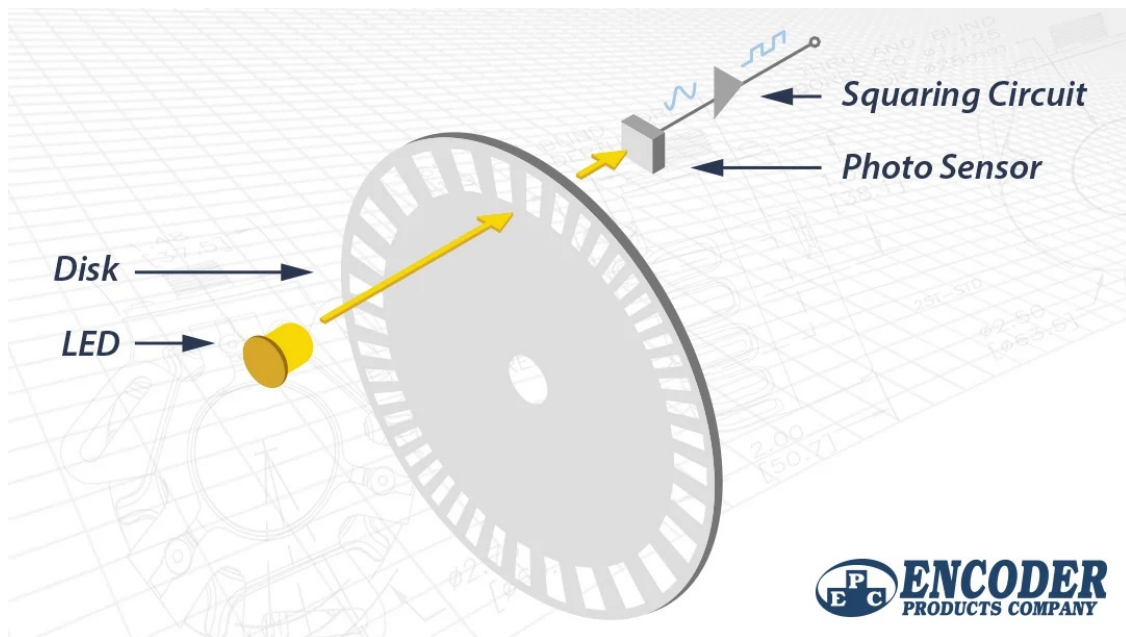
Dalším senzorem využívajícím logický vstup tak je jednoduchý enkodér (obr. 6.6), který slouží k určování rychlosti otáčení a lze použít k výpočtu změny polohy.



Obrázek 6.6: Enkodér v sadě Mechduino

Princip práce enkodéru je vidět na obr. 6.7, a spočívá v přerušování světelného toku pomocí disku, na kterém se v pravidelných intervalech střídají světlo propustné a světlo blokující oblasti. Jak se disk otáčí, tak se na světelném senzoru vytváří po zpracování obdélníkový signál. Rychlost otáčení lze pak vypočítat ze znalosti doby jedné periody a počtu snímaných oblastí na disku. Na disku použitým v sadě Mechduino je 16 oblastí, rychlost otáčení se počítá podle vzorce 6.1. Pokud by se používal disk s jiným počtem oblastí, je třeba pak výslednou rychlost v Matlabu přepočítat.

$$\omega = \frac{1}{N} \frac{1}{\Delta t} \cdot 60 = \frac{1}{16} \frac{1}{\Delta t} \cdot 60 \text{ [ot/min]} \quad (6.1)$$



Obrázek 6.7: Princip funkce enkodéru, převzato z [6]

Zadefinování třídy Encoder:

```

1  % zdefinovani pinu
2  encoder_pin = 20;
3  % zdefinovani enkoderu
4  encoder = my_arduino.get_Encoder(encoder_pin);

```

Rychlost otáčení pak lze získat pomocí následujícího příkazu, kde výsledná hodnota je v ot/min (RPM) a počítá se s diskem, který má 16 oblastí (16 děr):

```

1  speed = encoder.get_speed();

```

Použití jako senzor změny polohy

Jelikož enkodér má jen jeden signál, nelze z něj určovat směr otáčení, pouze uraženou vzdálenost v pulzech, neboli o kolik oblastí se senzor otočil. Pokud je senzor připojen na motor, lze směr otáčení brát dle směru otáčení motoru. Funkce vrátí počet pulzů enkodéru od posledního zavolání této funkce, je tedy třeba, pokud je volána až později v programu, tak ji před použitím zavolat na vymazání vnitřního bufferu.

```

1  ticks = encoder.get_ticks();

```

6 VSTUPNÍ PERIFERIE

Příklad použití

Uvedeme si dva příklady, u obou budeme řídit DC motor, který bude mít na výstupní hřídeli disk enkodéru.

U obou příkladů je shodné základní nastavení DC motoru:

```
1      com_port          = 'COM3';
2      encoder_pin       = 16;
3      motor_pin_A       = 12;
4      motor_pin_PWM     = 10;
5
6      % New Arduino controller
7      my_arduino = Mechduino(com_port);
8
9      % New DC motor and encoder controllers
10     encoder = my_arduino.get_Encoder(encoder_pin);
11     motor = my_arduino.get_DC_motor(motor_pin_A ,
        motor_pin_PWM);
```

V prvním příkladě je úkolem nastavehní otáček motoru na 100 RPM.

```
1      %% 1) set speed to 100 RPM
2      actual_motor_speed = 0;
3      motor.set_speed(actual_motor_speed);
4      motor.run();
5      while(encoder.get_speed() < 100)
6          % Displaying measured speed
7          speed = encoder.get_speed();
8          disp("Detected speed: " + speed + " RPM");
9          % increase motor speed
10         actual_motor_speed = actual_motor_speed + 1;
11         motor.set_speed(actual_motor_speed);
12     end
13     motor.stop();
```

6 VSTUPNÍ PERIFERIE

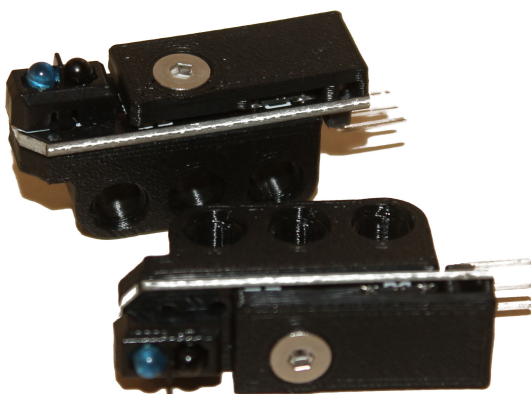
V druhém příkladě je úkolem změna směru motoru po uražení 100 dílků.

```
1      %% 2) after 100 ticks change direction
2      ticks = 0;
3      % reset ticks counter
4      encoder.get_ticks();
5      t = tic;
6      motor.run();
7
8      while(toc(t) < 30)
9          % Displaying measured distance
10         ticks = ticks + encoder.get_ticks();
11         disp("Ticks: " + ticks);
12         % change dir if distance is more than 99 ticks
13         if ticks > 99
14             motor.change_direction();
15             ticks = 0;
16         end
17         my_arduino.pause(50);
18     end
19     motor.stop();
```

6 VSTUPNÍ PERIFERIE

6.1.3 Sledování čáry

Senzor, používaný na sledování čáry (obr. 6.8), nebo rozlišení jakýchkoliv dvou situací, kdy jedna část má světlý a druhá tmavý povrch pracuje stejným způsobem jako přepínač, tedy vysílá logickou 0 pro neodrazivý povrch (černá barva, neosvětlený prostor, ...) a logickou 1 pro odrazivý povrch (bílá barva, zrcadlo, světelný zdroj, ...).



Obrázek 6.8: Senzor na sledování čáry v sadě Mechduino

Práce se senzorem je stejná jako práce s obecným logickým vstupem.

```
1      % zdefinovani pinu
2      sensor_pin  = 20;
3      % zdefinovani logickeho vstupu
4      line_sensor = my_arduino.get_Logic_input(sensor_pin);
5      line_detected = line_sensor.read()
6      % true pokud je nad bilou, false pokud je nad cernou
```

6.1.4 Indukční senzor

Indukční senzor se spíná při přiblížení kovového předmětu (vzdálenost v 0-4 mm). V sadě mechduino je indukční senzor typ TL-W5MC1 [7], který se sepne při detekci kovového předmětu - začne vysílat logickou 1, v rozeplém stavu vysílá logickou 0. senzor pracuje se všemi druhy kovů a magnety.

Zapojení senzoru

- hnědý drát - VCC
- modrý drát - GND
- černý drát - data

6 VSTUPNÍ PERIFERIE



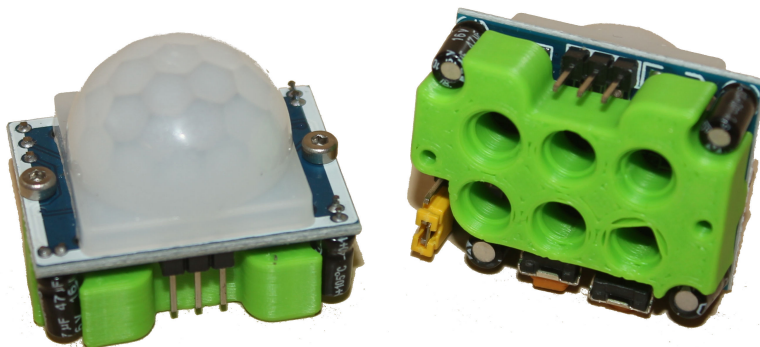
Obrázek 6.9: Indukční koncový spínač v sadě Mechduino

Práce se senzorem je stejná jako práce s obecným logickým vstupem.

```
1  % zdefinovani pinu
2  sensor_pin  = 20;
3  % zdefinovani logickeho vstupu
4  inductive = my_arduino.get_Logic_input(sensor_pin);
5  object_detect = inductive.read()
6  % true pokud je nad kovem, false pokud neni
```

6.1.5 Senzor pohybu

V sadě Mechduino jsou dva senzory pohybu založené na různých technologiích. Prvním z nich je PIR senzor (obr. 6.10), který pohyb určuje na základě změny teplotního pozadí před senzorem. Druhý senzor je založen na principu Dopplerova jevu, kdy vysílá signál o určité frekvenci a snímá odražený signál (obr. 6.11). Pokud má signál odlišnou frekvenci než vysílaný, znamená to změnu frekvence způsobenou pohybem.



Obrázek 6.10: PIR senzor pohybu v sadě Mechduino

6 VSTUPNÍ PERIFERIE



Obrázek 6.11: Senzor pohybu založen na Dopplerově jevu v sadě Mechduino

Oba senzory se z hlediska zapojení a zpracování dat chovají obdobně, při zaznamenání pohybu je na datovém pinu logická 1, jinak je tam logická nula.

```
1      % zdefinování pinu
2      sensor_pin = 20;
3      % zdefinování logického vstupu
4      motion_s = my_arduino.get_Logic_input(sensor_pin);
5      object_detect = motion_s.read()
6      % true pokud je nad kovem, false pokud není
```

6.2 Analogový vstup

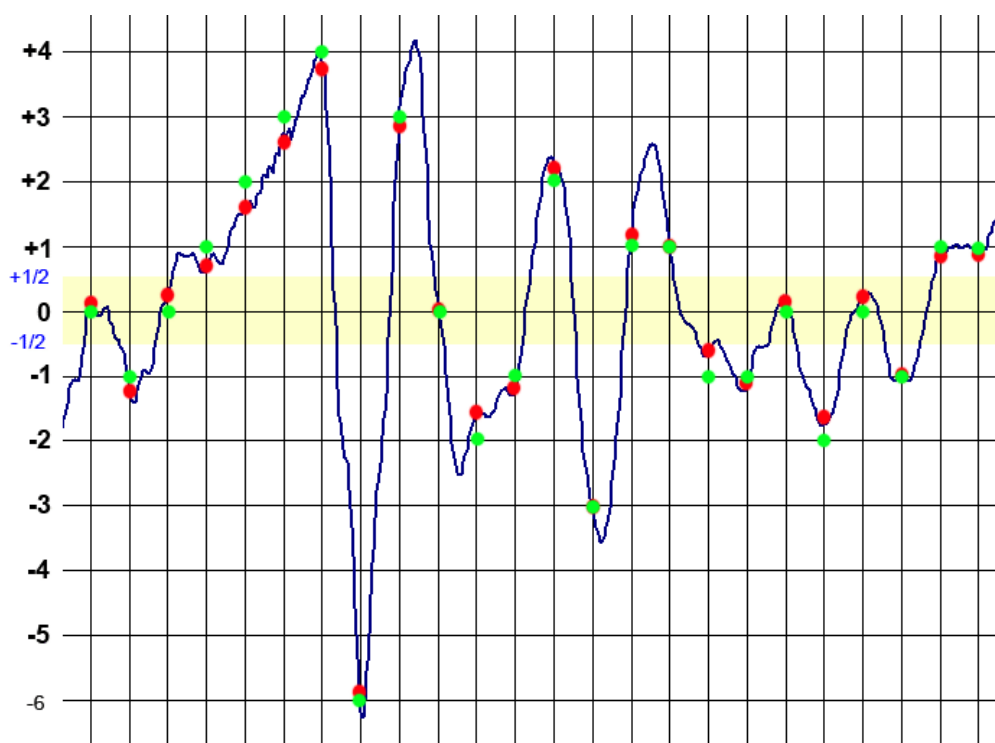
Vstupem je analogový signál, tedy spojitý signál v rozsahu $0 - U_{IN}$. Převod signálu probíhá ve dvou fázích, vzorkování a kvantování.

Vzorkování spočívá v rozdělení spojitého signálu na diskrétní dle časové osy. Tedy ze spojitého signálu se vždy po určitém časovém úseku, který je dán jako převrácená hodnota vzorkovací frekvence, vezme okamžitá hodnota (obr. 6.12 červené body).

Kvantování zpracovávaného signálu je nutné vzhledem k tomu, že digitalizovat signál můžeme pouze v určitém rozlišení, které je dáno parametry A/D převodníku (zobrazeno jako zelené body na obr. 6.12). Z principu funkce převodníku je rozlišení udáváno jako mocnina o základu 2 (2^N). Arduino na svých zařízeních používá 10 bit A/D převodník, tedy jeho rozlišení je $2^{10} = 1024$ a jeho rozsah napětí je $0 - 5\text{ V}$, kvantujeme tedy s rozlišením:

$$\frac{U_{IN}}{2^N - 1} = \frac{5}{2^{10} - 1} = 0,00489\text{ V}$$

6 VSTUPNÍ PERIFERIE



Obrázek 6.12: Princip vzorkování a kvantování, převzato z [8]

Použití

Pokud máme senzor, který vrací spojitou hodnotu napětí, lze jej měřit pomocí A/D převodníku, příkladem můžou být:

- Velikost napětí (pouze v rozsahu $0 - V_{MAX}$).
- Pomocí mikrofonu intenzitu hluku.
- Jednoduchý kapacitní senzor dotyku.

Používané příkazy

Zadefinování logického vstupu:

```
1  % zdefinovani pinu
2  sensor_pin = 20;
3  % zdefinovani logickeho vstupu
4  sensor = my_arduino.get_Analog_input(sensor_pin);
```

Zjištění vstupní hodnoty:

```
1  % vraci 0-1023 dle hodnoty na vstupu
2  sensor.get_value()
```

Zjištění prahové hodnoty pro přístup logický výstup:

6 VSTUPNÍ PERIFERIE

```
1      % nastaveni hodnoty pro treshold 0-1023
2      sensor.set_treshold(128)
3      % vraci true/false dle hodnoty na vstupu a
        nastaveného tresholdu
4      sensor.is_higher()
```

Příklad

Pro demonstraci použijeme mikrofon, jako analogový vstup a LED diodu pro vizuální kontrolu překročení nastavené hodnoty tresholdu, lze použít integrovanou LED diodu, která je připojena na pin 13. V příkladu se vypisuje intenzita zvuku po dobu 30 s a LED dioda svítí pokud není překročena nastavené hranice.

```
1      % zadefinovani pinu senzoru a LED
2      mic_pin  = 1; % pin A1, data pin 55
3      led_pin  = 13; % data pin 13
4      % zadefinovani LED a logickeho vstupu
5      microphone = my_arduino.get_Analog_input(mic_pin);
6      led = Arduino_MEGA_mini.get_LED(led_pin);
7
8      mic.set_treshold(100);
9      t = tic;
10     while(toc(t) < 30)
11         val = mic.get_value();
12         disp("Amplitude :" + val);
13         pause(0.1);
14         if(mic.is_higher())
15             led.turn_off();
16         else
17             led.light_up();
18         end
19     end
```


6 VSTUPNÍ PERIFERIE

6.2.1 Mikrofon

Mikrofon v sadě mechuino se je použitelný pouze na zjišťování intenzity hluku, hlavně z důvodu pomalé vzorkovací frekvence a nízkého rozlišení. Pro zpracování audiosignálu je nutné použít mikrofon připojený do počítače přes rozhraní Matlabu, více k tomuto tématu na:

`uk.mathworks.com/help/matlab/import_export/record-and-play-audio.html`

Příklad použití mikrofону je v předchozí kapitole, Analogový vstup.

6.2.2 Senzor světla

TODO

6.3 Komunikační sběrnice

Posledním způsobem komunikace periferie se zařízením je komunikační linka. Běžně se používají komunikační linky typu SPI, I²C, nebo sériová linka. Obecně se za sběrnici dá považovat vše, co má ustálenou komunikační strukturu. Sběrnice se mohou lišit počtem vodičů, množstvím napěťových hladin, hierarchií a rychlostí.

Z hlediska uživatelského je třeba dodržet pouze zapojení komunikační sběrnice, ostatní nastavení jsou již předem nastavena v knihovně Mechduino.

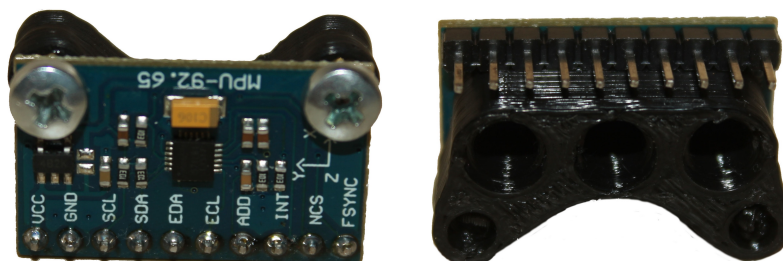
Přes některou z komunikačních sběrnic jsou připojeny tyto senzory:

- Akcelerometr s gyroskopem (SPI)
- Senzor hmotnosti (seriová linka)
- Magnetický enkodér (SPI)
- Senzor vzdálenosti na principu Lidaru (SPI)
- Senzor vzdálenosti na principu ultrazvuku (SPI)

Nevýhodou akcelerometru a gyroskopu je nestálost měření v čase, kdy měřené veličiny mají tendenci driftovat. Jedná se o jev, kdy i u naprosto stálého senzoru z měřených dat vypadá, že se pohybuje a otáčí. To se projevuje hlavně v případě, kdy chceme ze senzorů měřená data integrovat, tedy určovat ze senzorů pozici a natočení, kdy se tato chyba neustále nasčítává. Proto je dobré senzor kombinovat i jinými zdroji informací, které umožní korekci driftování.

6.3.1 Akcelerometr + Gyroskop

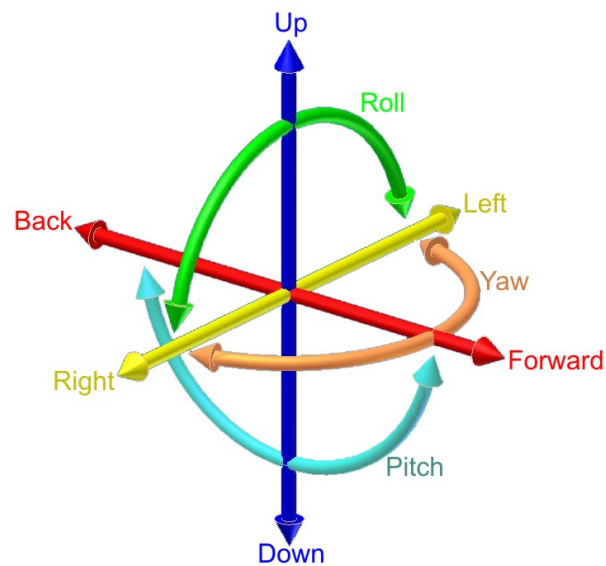
Akcelerometr měří přímočaré zrychlení, použitý akcelerometr snímá zrychlení ve všech třech osách (a_x , a_y , a_z). Gyroskop měří úhlovou rychlost, pro použitý senzor opět platí že snímá úhlovou rychlost ve třech osách (ω_x , ω_y , ω_z). Dohromady tvoří tyto pohyby šest stupňů volnosti v prostoru (obr. 6.14).



Obrázek 6.13: Akcelerometr s gyroskopem v sadě Mechduino

Akcelerometr komunikuje s Arduinem pomocí periferie SPI. Tato periferie komunikuje po dvou vodičích, kdy jeden přenáší časovou osu (SCL - na Arduinu pin 21) a druhý přenáší data (SDA - na Arduinu pin 20). Tyto piny je třeba propojit s příslušnými SCL a SDA piny na akcelerometru.

6 VSTUPNÍ PERIFERIE



Obrázek 6.14: Šest stupňů volnosti, převzato z [9]

Používané příkazy

Zadefinování akcelerometru s gyroskopem:

```
1 % zdefinovani logickeho vstupu
2 sensor = my_arduino.get_Gyroscope();
```

Zjištění vstupní hodnoty:

```
1 % gyroskop vraci naklon ve trech osach
2 [roll, pitch, yaw] = sensor.get_angles();
3 % akcelerometr vraci zrychleni ve trech osach
4 [AccX, AccY, AccZ] = sensor.get_acc();
```

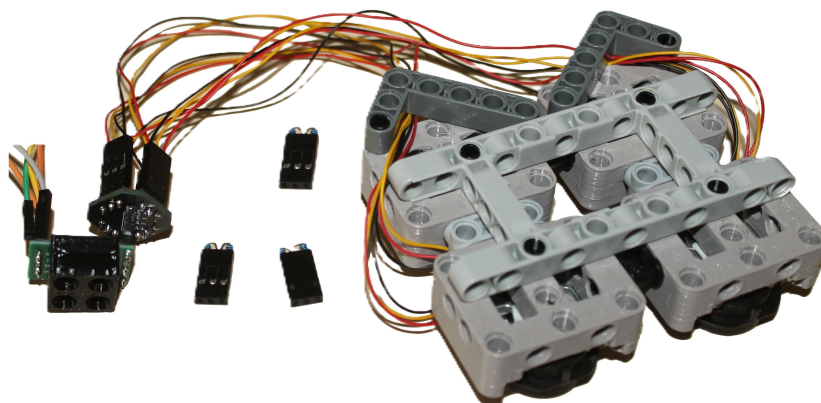
Z důvodu problému driftování hodnot je vytvořena funkce na resetování hodnot náklonu gyroskopu, tato funkce zároveň nastavuje úhel do nulových hodnot a je nutné ji zavolat na začátku programu po zdefinování senzoru.

```
1 % Initial calibration of sensor
2 sensor.calibrate();
```

6 VSTUPNÍ PERIFERIE

6.3.2 Senzor hmotnosti

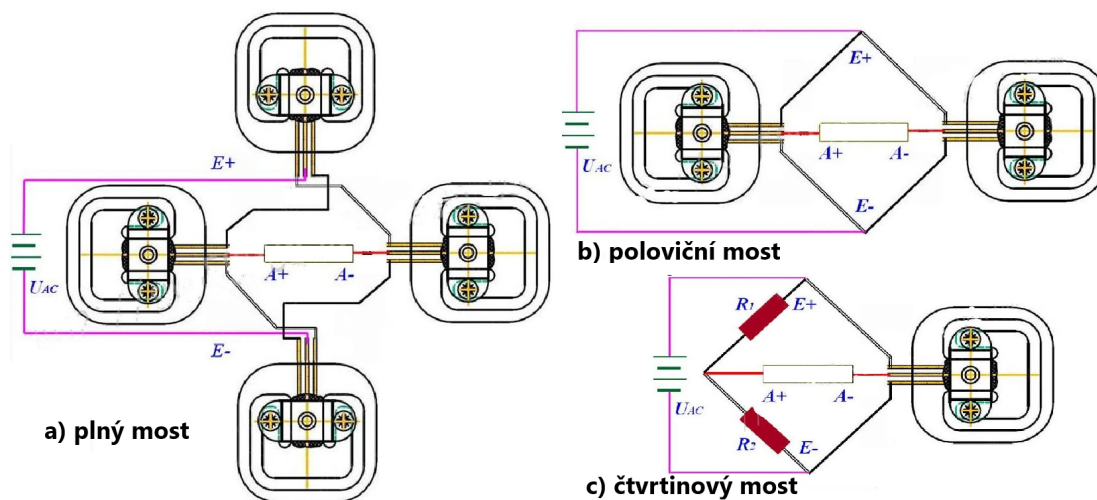
Senzor hmotnosti pracuje na principu měřené přetvoření pomocí tenzometrů, v sadě mechuino jsou v základu 4 moduly GUANG CE YZC-161 [10], kde každý váží do 50 kg a 24 bitový ADC převodník HX711 [11] (na obr. 6.15).



Obrázek 6.15: Vaha v sadě Mechduino

Zapojení:

ADC převodník HX111 má dva vstupní kanály (A a B) ale z důvodu vyššího rozlišení se používá pouze kanál A. Tenzometry se pro zapojení do váhy zapojují do wheastnova mostu, který slouží pro měření malých rozdílů mezi podobnými odpory (obr. 6.16).

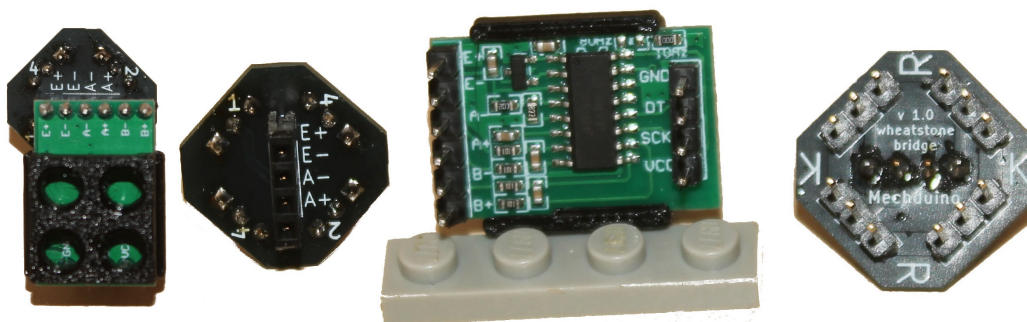


Obrázek 6.16: vaha zapojení

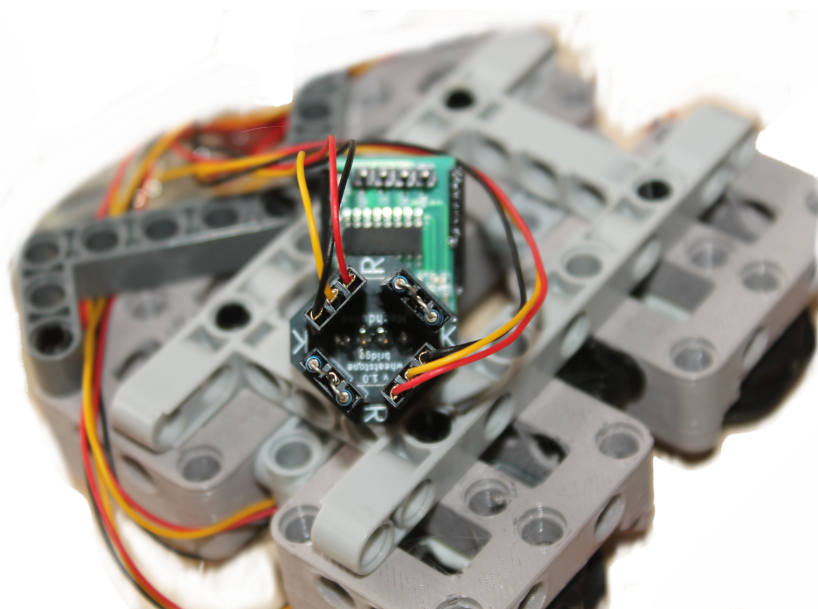
Pro usnadnění zapojení je v sadě propojovací destička, která se připojí mezi modul HX711 a tenzometrické snímeče váhy (obr. 6.17). Na horní straně ismena K a R značí pozici černých (K) a červených (R) vodičů z jednotlivých modulů váhy. Čísla na druhé straně desky značí pořadí zapojování senzorů pro různé druhy zapojení, tedy pro 1/4 most zapojíme tenzometr na konektor 1 a na zbylé se zapojí zkratovací propojky. Pro 1/2 most zapojíme tenzometry na pozice 1 a 2, a pro celý

6 VSTUPNÍ PERIFERIE

most zapojíme všechny 4 pozice. Příklad zapojení 1/2 mostu je vidět na obr. 6.18.



Obrázek 6.17: detail modulů váhy v sadě Mechduino



Obrázek 6.18: příklad zapojení polovičního mostu

Používané příkazy

Zadefinování senzoru hmotnosti:

```
1 % zdefinovani logickeho vstupu  
2 vaha = my_arduino.get_Mass_sensor(CLK_pin , DATA_pin);
```

Nastavení zesílení:

```
1 vaha.set_scale(-2.767);
```

6 VSTUPNÍ PERIFERIE

Jak zjistit zesílení:

- Nastavíme zesílení na 1.
- Nachystáme si kalibrační předmět o známé hmotnosti - m_{znama}
 - Těžší těleso má větší odstup od šumu -> přesnější výsledek
- Zapneme váhu, vynulujeme a zvážíme kalibrační předmět - m_{merena}
- Výsledné zesílení je:

$$scale = \frac{m_{merena}}{m_{znama}}$$

- Výsledná váha vyjde v jednotkách ve kterou váhu zadáváme
- Výsledek je zaokrouhlen na celé číslo
- Pokud $scale$ vyjde větší než 10^N lze $scale$ vydělit 10^N pro získání přesnosti na N desetinných míst.
- Příklad pro závaží o hmotnosti 1000 g:

$$scale = \frac{m_{merena}}{m_{znama}} = \frac{27670}{1000} = 27.67$$

- 27.67 je větší než 10, vydělením 10 lze získat přesnost 0.1 g.
- Při použití tenzometrů v sadě mechuino a rozlišení 0.1 g je obvyklá hodnota $scale = 2.767$.

Opětovné nastavení nulové hodnoty:

```
1 vaha.set_zero(); % set new zero point
```

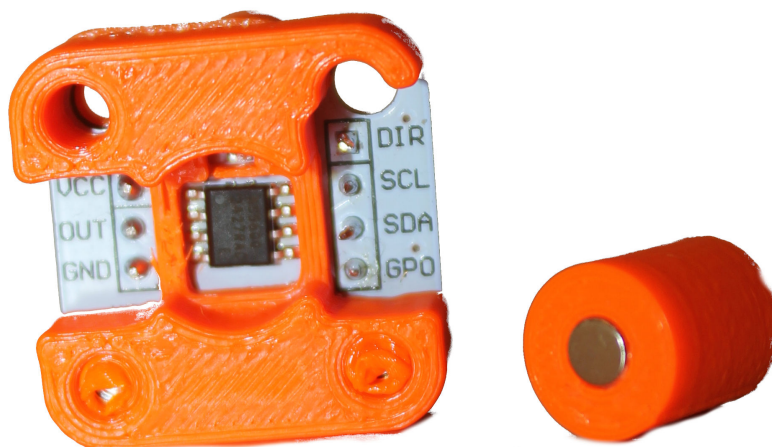
Čtení zvážené hodnoty, velikost je závislá na hodnotě zesílení:

```
1 mass = vaha.get_value();
```

6.3.3 Magnetický enkodér

Magnetický enkodér je absolutní snímač polohy pro jednu otáčku, který je schopen určit polohu s přesností 2^{12} , tedy 4096 pozic na otáčku. Senzor pracuje na principu snímání magnetického pole permanentního magnetu v prostoru, proto je důležité co nejpřesněji dodržet pozici a vzdálenost senzoru a magnetu. V sadě Mechduino je to částečně zajištěno fyzickou konstrukcí obalu senzoru a magnetu.

6 VSTUPNÍ PERIFERIE



Obrázek 6.19: Magnetický enkodér v sadě Mechduino

Zapojení:

Magnetický enkodér komunikuje s Arduinem pomocí periférie SPI. Tato periférie komunikuje po dvou vodičích, kdy jeden přenáší časovou osu (SCL - na Arduinu pin 21) a druhý přenáší data (SDA - na Arduinu pin 20). Tyto piny je třeba propojit s příslušnými SCL a SDA piny na enkodéru.

Používané příkazy

Zadefinování magnetického enkodéru:

```
1 % zdefinovani magnetickeho encoderu
2 encoder = my_arduino.get_Magnetic_encoder();
```

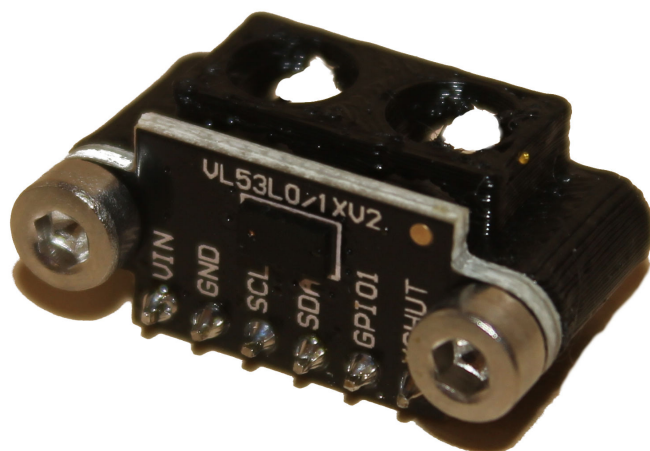
Úhel vrámci jedné otáčky lze vyčíst ve stupních, v radiánech, nebo v RAW hodnotě 0-4095:

```
1 angle_deg = encoder.get_angle_deg();
2 angle_rad = encoder.get_angle_rad();
3 angle_raw = encoder.get_angle_raw();
```

6.3.4 Senzor vzdálenosti - Lidar

Pro měření vzdálenosti na bázi lidarů je použit senzor VL53LO, který pracuje na principu měření vzdálenosti pomocí ToF (Time-of-Flight). Senzor dokáže spolehlivě měřit vzdálenost v rozmezí 20 - 700 mm.

6 VSTUPNÍ PERIFERIE



Obrázek 6.20: Lidar v sadě Mechduino

Zapojení:

Lidar komunikuje s Arduinem pomocí periferie SPI. Tato periferie komunikuje po dvou vodičích, kdy jeden přenáší časovou osu (SCL - na Arduinu pin 21) a druhý přenáší data (SDA - na Arduinu pin 20). Tyto piny je třeba propojit s příslušnými SCL a SDA piny na lidar.

Používané příkazy

Zadefinování lidarů:

```
1 % zdefinovani lidarů  
2 lidar = my_arduino.get_Lidar();
```

Získání změřené vzdálenosti:

```
1 distance = lidar.get_distance(); % [mm]
```

6.3.5 Senzor vzdálenosti - Ultrazvuk

Pro měření vzdálenosti na bázi ultrazvuku je použit senzor, který pracuje na principu měření vzdálenosti pomocí na základě času doby letu odraženého ultrazvuku, tedy času než se senzorem vyslaný signál odražený od překážky vrátí zpět (obr. 6.21).



Obrázek 6.21: Ultrazvukový senzor v sadě Mechduino

6 VSTUPNÍ PERIFERIE

Zapojení:

Ultrazvuk komunikuje s Arduinem pomocí dvou vodičů, které jsou označeny trig a echo, pro tyto vodiče je nutné při spuštění programu zdefinovat, kam jsou připojené.

Používané příkazy

Zadefinování lidarů:

```
1      % zdefinovani ultrazvuku
2      trig_pin = 7;
3      echo_pin = 8;
4      sonar = my_arduino.get_Ultrasonic_sensor(trig_pin,
          echo_pin);
```

Získání změřené vzdálenosti:

```
1      distance = sonar.get_distance(); % [mm]
```

Seznam obrázků

2.1	Menu aplikací v Matlab 2024a	4
2.2	Vyhledání toolboxu Mechduino v Add-ons explorer	4
2.3	Instalace toolboxu Mechduino	4
2.4	Panel aplikací	5
4.1	Popis Arduino MEGA mini 2560, upraveno z [1]	8
5.1	Příklad výstupních periférií	9
5.2	modul LED semaforu	11
5.3	Popis PWM signálu	13
5.4	DC motor s driverem L298N	15
5.5	Zapojení driveru L298N, převzato z [2]	16
5.6	Zapojení driveru L298N	17
5.7	krokový motor s driverem	20
5.8	modelářský servo motor, převzato z [5]	22
6.1	Příklad vstupních periférií	23
6.2	Schéma zapojení logického vstupu	25
6.3	Tlačítko v sadě Mechduino	26
6.4	Schéma zapojení tlačítka	26
6.5	Náběžná a sestupná hrana signálu	27
6.6	Enkodér v sadě Mechduino	27
6.7	Princip funkce enkodéru, převzato z [6]	28
6.8	Senzor na sledování čáry v sadě Mechduino	31
6.9	Indukční koncový spínač v sadě Mechduino	32
6.10	PIR senzor pohybu v sadě Mechduino	32
6.11	Senzor pohybu založen na Dopplerově jevu v sadě Mechduino	33
6.12	Princip vzorkování a kvantování, převzato z [8]	34
6.13	Akcelerometr s gyroskopem v sadě Mechduino	37
6.14	Šest stupňů volnosti, převzato z [9]	38
6.15	Vaha v sadě Mechduino	39
6.16	vaha zapojení	39
6.17	detail modulů váhy v sadě Mechduino	40
6.18	příklad zapojení polovičního mostu	40
6.19	Magnetický enkodér v sadě Mechduino	42
6.20	Lidar v sadě Mechduino	43
6.21	Ultrazvukový senzor v sadě Mechduino	43

Seznam zdrojů a použité literatury

- [1] *Arduino mega 2560 mini*. Dostupné také z: <https://www.electronicparts.com/products/arduino-mega-2560-pro-embedded-mcu-mini-atmega2560-micro-usb-ch340g-2018>.
- [2] *L298N Driver*. Dostupné také z: <https://novatronicec.com/index.php/product/l298n-driver-para-motor-dc/>.
- [3] *Krokový motor 28BYJ-48*. Dostupné také z: <https://www.laskakit.cz/krokovy-motor-28byj-48/>.
- [4] *Řadič ULN2003 pro krokový motor*. Dostupné také z: <https://www.laskakit.cz/radic-uln2003-pro-krokovy-motor/>.
- [5] *Servo motor*. Dostupné také z: <https://dratek.cz/arduino/123001-servo-mg996r-mg996-s-plastovymi-prevody-13kg-180.html>.
- [6] *What Is an Optical Encoder?* 2018. Dostupné také z: <https://www.encoder.com/article-what-is-an-optical-encoder>.
- [7] *Indukční senzor TL-W5MC1*. Dostupné také z: <https://www.laskakit.cz/bezdotykovy-indukcni-snimac-tl-w5mc1-npn-dc-6-36v/>.
- [8] *Wikipedia ADC* [online]. [cit. 2025-06-30]. Dostupné z: https://cs.wikipedia.org/wiki/A/D_p%C5%99evodn%C3%ADk.
- [9] *Wikipedia Gyroskop* [online]. [cit. 2025-06-30]. Dostupné z: https://simple.wikipedia.org/wiki/Pitch,_yaw,_and_roll.
- [10] *GUANG CE YZC-161 Vážicí senzor 50kg*. [B.r.]. Dostupné také z: <https://www.laskakit.cz/guang-ce-yzc-161-vazici-senzor-50kg/>.
- [11] *AD Převodník Modul 24-bit 2 kanály HX711*. Dostupné také z: <https://www.laskakit.cz/ad-prevodnik-modul-24-bit-2-kanaly-hx711/>.